



Provenance-Aware Impermanent Loss Protection

A Uniswap v4 hook that converts content-authenticity data into an on-chain Dilution Risk Score, then uses it to gate IP tokenization and calibrate LP fee protection across the full creator lifecycle.

EVENT	NETWORK	TEAM	SUBMISSION
UHI9 Hookathon	Unichain Sepolia (1301)	Scientivan · 2026	bit.ly/veritas-submission-files



Creators can't monetize. LPs can't invest.

Five structural failures. The infrastructure is missing.

\$250B+

Creator economy size; no on-chain capital infrastructure exists for it

Goldman Sachs · 2023

\$180B

Global IP licensing revenue per year, mostly captured by intermediaries

WIPO / ICC · 2024

~50%

Uniswap v3 LPs earned negative returns due to impermanent loss

Bancor divergence-loss study · 2022

\$8.9B

NFT wash-trading volume in 2022; no origination gate ever existed

Chainalysis Crypto Crime · 2023

01	Creators locked out of their own IP revenue	The \$250B creator economy has no trustless capital rail. Royalties route through intermediaries who take the majority. No on-chain mechanism lets a creator raise capital from original work without ceding control.
02	No fair-launch standard for IP tokenization	Existing launchpads allow insider allocation, rug pulls, and no enforced royalties. Creators who tokenize their work have no trustless protection after the initial sale.
03	Copied and AI content floods the market	Anyone can tokenize a copy, an AI replica, or plagiarized work. No on-chain gate blocks low-originality content from raising capital alongside genuine originals.
04	IL risk from content dilution is invisible on-chain	As copies multiply, the underlying asset loses scarcity. That loss is the main driver of price divergence in a content-asset pool. The AMM cannot see it, so it cannot price it.
05	LPs cannot enter a market they cannot underwrite	Without a priced dilution risk, liquidity providers have no rational basis to commit capital to content pools. TVL in this category is essentially zero. No creator can graduate to deep liquidity.

FIVE FAILURES. ONE SYSTEM.

Two broken markets. One missing primitive.

A trustless launchpad for creators. A live dilution signal for LPs.
The same system. One score connects them.

● System V1: IP Launchpad

● Bridge: IPLaunchRegistry

● System V2: DRS Oracle

V4 hooks enable what v3 made impossible.

Three hook permissions. Three structural problems solved. One deployment.

beforeSwap + BeforeSwapDelta

Bonding Curve

The hook intercepts every swap and returns a BeforeSwapDelta that overrides AMM pricing entirely. The PoolManager treats it as a valid trade while the hook runs the quadratic fair-launch curve. No separate liquidity pool needed during the raise phase. Impossible in v3 without a custom wrapper contract.

SYSTEM V1

afterSwap

Perpetual Royalties

Every swap on the graduated v4 pool triggers afterSwap. The hook routes 0.20% of the swap amount to the creator's pending balance on every trade. No separate royalty contract. No intermediary. No opt-out. The creator earns on every secondary trade, forever, enforced at the protocol level.

SYSTEM V1

beforeSwap + DYNAMIC_FEE

Live IL Protection

getHookPermissions sets the DYNAMIC_FEE flag. The PoolManager calls the hook for the fee on every swap. The hook reads getCurrentDRS() from VeritasRegistry. Fee rises as dilutionCount rises. No redeployment. No governance vote. IL protection adjusts automatically to the content's real-world dilution state.

SYSTEM V2

IP Tokenization Launchpad

- 1

Mint ERC-20 IP Token

Creator deploys a named token via IPTokenFactory. Fixed supply: 60% curve / 30% LP / 10% creator. No insider allocation possible.

```
IPTokenFactory.createToken(name, symbol, supply)
```
- 2

Open Bonding Curve

VeritasHook v1 opens a quadratic fair-launch curve. Price rises as collectors buy. AMM math bypassed entirely via BeforeSwapDelta.

```
VeritasHook.initializeLaunchpad(poolId, hardCap)
```
- 3

Graduation: curve closes, v4 pool opens

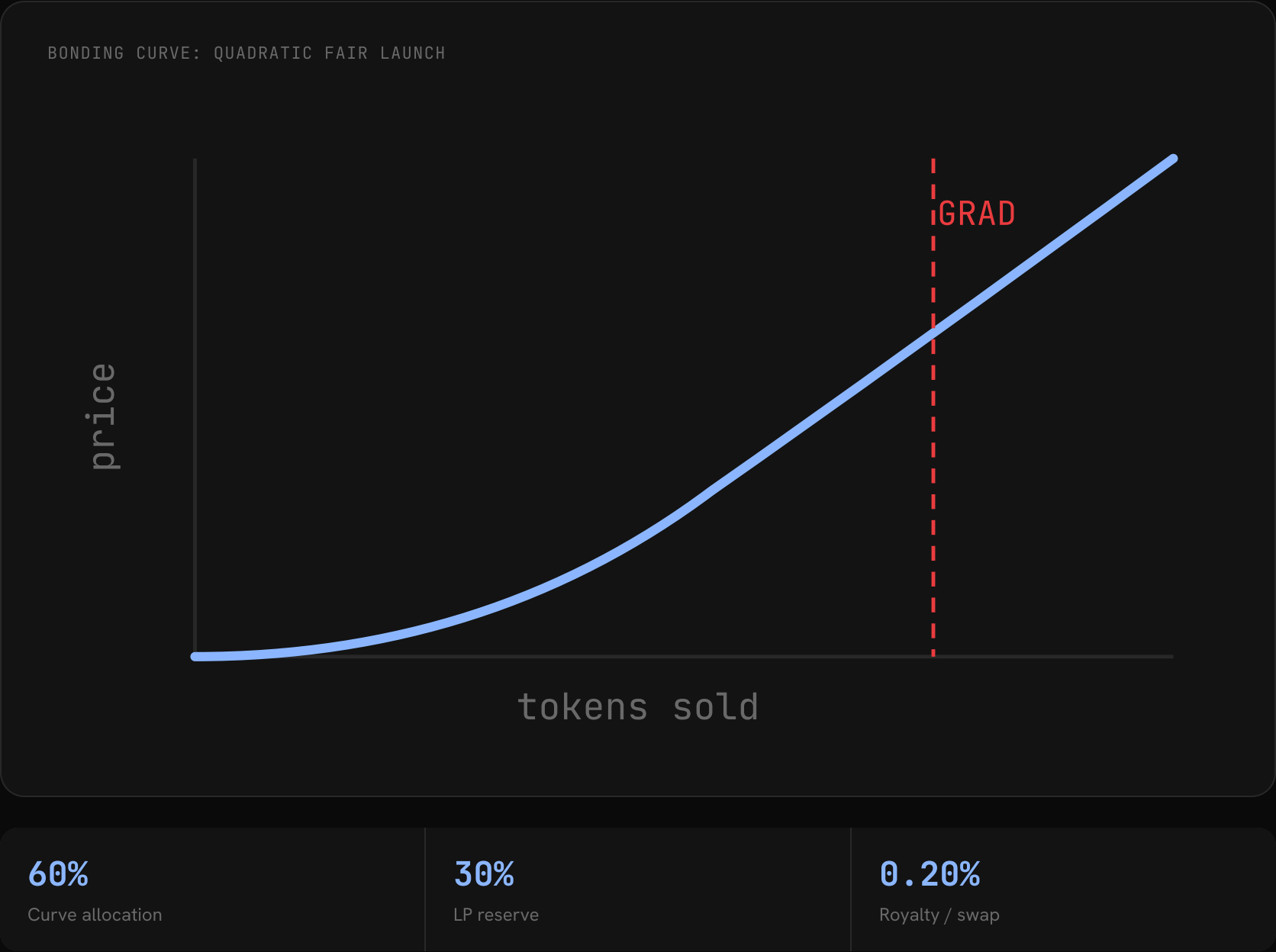
When hardCap is hit, liquidity locks permanently into a Uniswap v4 dynamic-fee pool. LP is protocol-owned; no rug possible.

```
Phase: LAUNCHPAD → GRADUATED → v4 pool active
```
- 4

Ongoing royalties

0.20% of every swap on the graduated v4 pool routes to the creator. Enforced inside afterSwap. No opt-out, no bypass, perpetual.

```
VeritasHook.pendingRoyalties[creator]
```



Dilution Risk Score Oracle

// noisy-OR • computed off-chain, verified on-chain

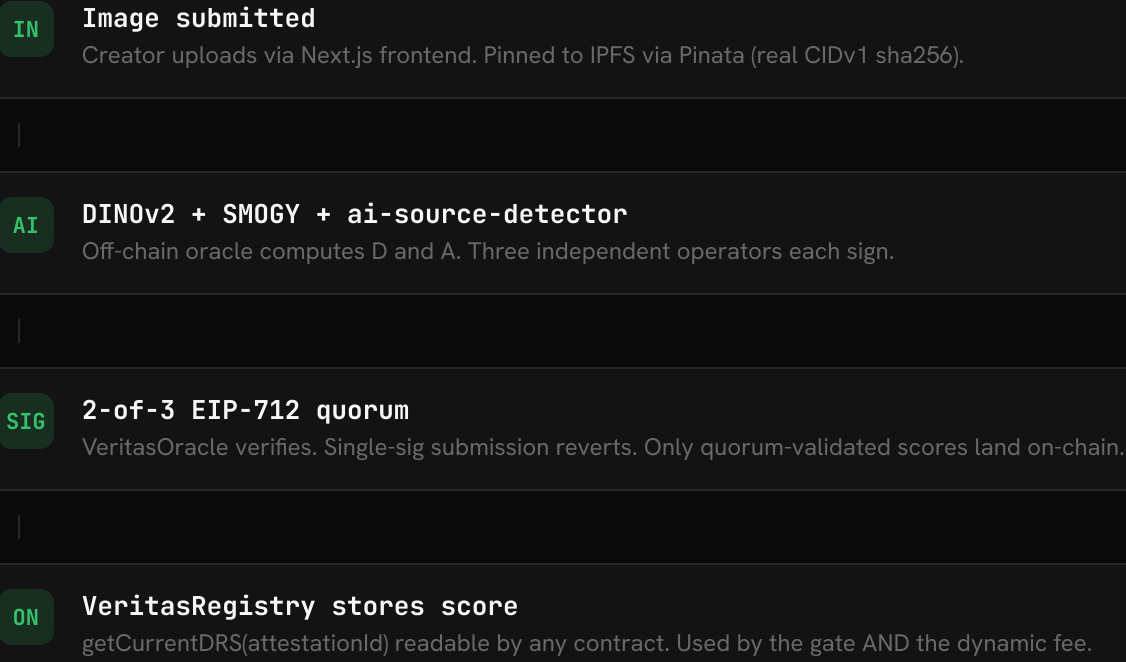
DRS = 1 - (1-D)(1-A)

D Duplicate Density
DINOv2 cosine similarity vs sqlite-vec corpus. Rises as near-copies appear. Autonomously updated by Living DRS; no keeper needed.

A AI Replicability
Ensemble: SMOGY (primary) + ai-source-detector (5-class: SD/MJ/DALLE/real/other). Measures how replicable the content is by current generative tools.

Mekacher et al., Scientific Reports 2022. Rarer assets carry lower IL. Peer-reviewed grounding for the fee model.

ORACLE PIPELINE



V1 and V2 do not run in parallel.

V2 is the prerequisite and the ongoing guardian of V1.

SYSTEM V2

DRS Oracle

Computes DRS off-chain, stores it on-chain. Makes the content's authenticity risk readable by any contract.

- > DINOv2 duplicate scan
- > AI replicability ensemble
- > 2-of-3 oracle quorum
- > `getCurrentDRS()` on VeritasRegistry



IPLAUNCHREGISTRY

0x9dF983...ea2

registerIP() reverts if
DRS $\geq$ 85%



After graduation, V2 still active.
getCurrentDRS() drives dynamic LP fee.

SYSTEM V1

IP Launchpad

Raises capital via bonding curve and graduates into a locked Uniswap v4 pool with ongoing royalties.

- > Mint ERC-20 IP token
- > Open fair-launch curve
- > Graduate to v4 pool
- > VeritasHook v2: fee = f(DRS)

The LP fee is not a setting. It is a function.

Fee rises linearly with DRS. IL risk is priced on every swap, automatically.

// computed on-chain per swap, reads live registry

$$\text{fee} = \text{base} + (\text{max} - \text{base}) \times \text{DRS}$$

- base** 0.30% (3000 bps · minimum fee)
- max** 1.00% (10000 bps · maximum fee)
- gate** DRS ≥ 85% → registerIP() reverts

DRS 0% **0.30%**

Original content. No copies in corpus. Minimum IL exposure. Minimum fee.

DRS 30% **0.51%**

Duplicate density rising. Scarcity beginning to erode. Fee proportionally elevated.

DRS 60% **0.72%**

High dilution. LP IL exposure is material. Fee compensates the additional risk.

DRS ≥ 85% **GATE**

IPLaunchRegistry reverts DRSTooHigh. This pool was never created. Market protected.

The implementation speaks for itself.

Three contracts. Three key functions. One coherent system.

DRS FORMULAVERITASREGISTRY.SOL

```
// noisy-OR combination: P(both safe) = (1-D)(1-A)
function getCurrentDRS(
    bytes32 id
) external view returns (uint256) {
    uint256 d = saturatedD(
        records[id].dilutionCount
    );
    return 10000 - (
        (10000 - d) *
        (10000 - records[id].aiScore)
    ) / 10000;
}
```

IP GATEIPLAUNCHREGISTRY.SOL

```
// Only original content can raise capital
function registerIP(
    bytes32 attestationId
) external {
    uint256 drs = registry.getCurrentDRS(
        attestationId
    );
    if (drs ≥ 8500)
        revert DRSTooHigh(drs);

    _register(attestationId, msg.sender);
}
```

DYNAMIC LP FEEVERITASHOOK.SOL

```
// IL protection: fee rises with DRS automatically
function _getDynamicFee(
    PoolId id
) internal view returns (uint24) {
    uint256 drs = registry.getCurrentDRS(
        poolToAttestation[id]
    );
    return uint24(baseFee +
        (maxFee - baseFee) *
        drs / 10000);
}
```

One score. Three actors. One lifecycle.

ROLE 01

Creator

Submits content, gets DRS attested, mints IP token, and opens the bonding-curve launchpad. Capital flows to them via royalties after graduation.

- V2** Attests content, gets DRS score from oracle
- GATE** registerIP() checks DRS < 85% before IP goes live
- V1** Mints ERC-20, opens bonding curve, earns royalties

ROLE 02

Collector

Buys IP tokens on the bonding curve. DRS is the trust signal before any purchase. Price discovery happens on the curve; at graduation, liquidity locks.

- V2** DRS shown as trust signal before buying
- V1** Buys on quadratic curve via VeritasHook v1

ROLE 03

LP

Provides liquidity to graduated v4 dynamic-fee pools. The DRS-calibrated fee compensates for IL. As D rises autonomously, the fee rises automatically.

- V2** Dynamic fee = f(getCurrentDRS). Living DRS keeps it current
- V1** Provides liquidity to graduated pool, earns fees

The full stack.

ON-CHAIN: UNICHAIN SEPOLIA (1301)

VeritasOracle

VeritasRegistry

IPLaunchRegistry

VeritasHook v1

VeritasHook v2

VeritasRegistryCallback

OFF-CHAIN ORACLE SERVICE

DINOv2 Embedder

SMOBY + ai-source-detector

2-of-3 Operator Quorum

IPFS via Pinata

DilutionMonitorRSC



D updates itself. No keeper.

- 1

Near-duplicate is attested

A second creator attests content with matching perceptual hash. VeritasRegistry emits NewAttestation.
event NewAttestation(bytes32 pHash, bytes32 attestationId)
- 2

RSC detects it cross-chain

DilutionMonitorRSC on Reactive Lasna is subscribed. react() fires immediately, zero latency.
0xB44F024...5C5d · Lasna (5318007)
- 3

Callback increments dilutionCount

RSC emits Callback to VeritasRegistryCallback on Unichain. incrementDilutionCount fires.
0xa516891...332a · Unichain Sepolia
- 4

LP fee rises automatically

saturatedD(dilutionCount) raises effectiveD. getCurrentDRS is higher. VeritasHook reads it on the next swap.

// Proven live on testnet · 2026-06-06

BEFORE COPY ATTESTED

DILUTION COUNT

0

EFFECTIVE D

0.00

DYNAMIC FEE

base

AFTER COPY ATTESTED

DILUTION COUNT

2

EFFECTIVE D

0.60

DYNAMIC FEE

elevated

• Proven live · DilutionIncremented on-chain

Original: 0x7d07e4ea...0a6
Copy: 0x0547799f...21c

It works.



TESTS PASSING

Full Foundry suite: unit + integration + **ReactiveIntegration.t.sol** (8 cross-chain tests). Registry coverage 99%.

Run: `forge test -vv`



CONTRACTS LIVE

Unichain Sepolia: VeritasOracle `0x45b3...4bd3` , VeritasRegistry `0xe359...d824` , VeritasHook `0x0Aaa...60C4` ,
VeritasRegistryCallback `0xa516...332a` , IPLaunchRegistry `0x9dF9...ea2` .

2/3

ORACLE QUORUM

Three independent operators, each signs DRS separately. **Single-sig attest reverts on-chain** (proven by test).
No centralized oracle risk.

cross-chain

REACTIVE NETWORK RSC

DilutionMonitorRSC live on **Lasna (5318007)**. Callback raised dilutionCount 0 to 2 in one near-duplicate test;
no cron, no keeper, no trust.

Built for the rubric.

CRITERIA	WEIGHT	WHAT WE BUILT
Original Idea	30%	First v4 hook to price IL from off-chain content-authenticity data. DRS = noisy-OR(D, A) is novel in DeFi. Peer-reviewed grounding (Mekacher 2022).
Unique Execution	25%	Living D via Reactive Network RSC (no keeper), 2-of-3 quorum, bonding-curve graduation, DRS gate, IL simulator. Two integrated systems, not one hook.
Impact	20%	Unlocks the full creator economy lifecycle currently impossible to build safely: verified origination, fair-launch capital, trustworthy LP, all from one score.
Functionality	15%	81 tests, 5 live contracts, proven cross-chain, real oracle with on-chain attest, real swaps on Unichain Sepolia. Not a demo: a working system.
Presentation	10%	This deck (17 slides, full narration). A separate demo video covering all four flows: Creator, Collector + LP, Living DRS, The Gate. On-chain proof replaces claims.

Deployed. Verified. Live.

V2 ORACLE SYSTEM

VeritasOracle 0x45b327808f...4bd3 2-of-3 EIP-712 quorum verifier. Single-sig reverts.
VeritasRegistry 0xe359bdB2cA...d824 Stores A + D + dilutionCount. Exposes getCurrentDRS().
VeritasRegistryCallback 0xa516891eE1...332a Reactive callback receiver. Wired as dilutionUpdater.
VeritasHook v2 0x0AaAef1B31...60C4 Dynamic LP fee = f(DRS). IL protection live on every swap.

V1 LAUNCHPAD SYSTEM

IPTokenFactory 0xFB112b3AdF...638c Deploys creator ERC-20 IP tokens. Fixed 60/30/10 split.
VeritasHook v1 0xcecf19e772...20cc Bonding curve + graduation + 0.20% perpetual royalties.
VeritasRegistry v1 0x17c5e35D11...fC8 Permissionless IP registry read by the bonding curve hook.
Mock USDC (raise token) 0xD977AD0334...15a Testnet USDC for launchpad buyers. Mintable on frontend.

BRIDGE

IPLaunchRegistry 0x9dF98317b0...ea2 DRS gate: registerIP() reverts if DRS ≥ 85%. The V1/V2 bridge contract.

POOLMANAGER

Uniswap v4 PoolManager 0x00B036B58a...62AC Official Unichain Sepolia deployment.

REACTIVE NETWORK

DilutionMonitorRSC 0xB44F024468...5C5d Subscribes to NewAttestation. Cross-chain callback with no keeper. Lasna (5318007)

LIVE POOLS

Forest Scene (DRS 0) poolId 0xf937...ff0e Base fee 0.30%. Original content, no dilution.
Living-D Original (DRS 0.68) poolId 0x0cb6...0316 Elevated fee. dilutionCount raised 0→2 by RSC live.

Run the demo.

Read the contracts.

Judge by what is on-chain.

Everything claimed here has a transaction hash.

RUN IT YOURSELF

- 1

```
cd oracle && npm run dev
```

Oracle service starts at localhost:8787. DINOv2 + SMOGY + ai-source-detector live.
- 2

```
cd frontend && npm run dev -- --webpack
```

Frontend at localhost:3000. Connect wallet to Unichain Sepolia (chainId 1301).
- 3

Navigate to `/launch` → click "Unique"

Watch real DRS score arrive from a live oracle. 2-of-3 badge confirms quorum. Attest on-chain.
- 4

```
forge test -vv
```

81 tests pass. ReactiveIntegration.t.sol (8 cross-chain tests) included.

VERIFY ON-CHAIN

- REG

VeritasRegistry (source-verified)

0xe359bdB2cA1A152Ae62E2496F140ea192Ce0d824
- HK

VeritasHook v2 (source-verified)

0x0AaAef1B312243EFaAD238fb53403dC5761d60C4
- RSC

DilutionMonitorRSC (Lasna 5318007)

0xB44F024468dc78572D1Ad7b3f5Ce3A51408E5C5d
- V1

VeritasHook v1 bonding curve

0xcecf19e7722e3b38b399785e00e13f7c3dcd20cc
- GH

GitHub · contracts + oracle + frontend

github.com/Scientivan/VeritasProtocolv2



Two systems. One score. The missing risk infrastructure
for the **creator economy**.

SYSTEM V1

IP Launchpad + Bonding Curve

SYSTEM V2

DRS Oracle + Dynamic Fee

BRIDGE

IPLaunchRegistry DRS Gate

TESTS

81 passing

CHAIN

Unichain Sepolia 1301

SUBMISSION FILES

bit.ly/veritas-submission-files

● Live on Unichain Sepolia · VeritasOracle / Registry / Hook source-verified